Report

Node description

```
 Model name:                    Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz
CPU(s):                         80

Server name:                    ProLiant XL270d Gen10

NUMA node(s):                   2
NUMA node0 CPU(s):              0-19,40-59
NUMA node1 CPU(s):              20-39,60-79

node 0 size:                    385636 MB
node 0 free:                    105947 MB

node 1 size:                    387008 MB
node 1 free:                    232013 MB

OS:                             Ubuntu 22.04.5 LTS
```

Scaling

Matrix size (M=N)	Threads														
	1	2		4		7		8		16		20		40	
	T(1), s	T(2), s	S(2)	T(4), s	S(4)	T(7), s	S(7)	T(8), s	S(8)	T(16), s	S(16)	T(20), s	S(20)	T(40), s	S(40)
20 000 (~3 GiB)	~1.06	~0.62	~1.71	~0.32	~3.31	~0.19	~5.57	~0.17	~6.23	~0.09	~11.7	~0.08	~13.25	~0.05	~21.2
40 000 (~12 GiB)	~4.27	~2.41	~1.77	~1.26	~3.39	~0.73	~5.85	~0.65	~6.57	~0.33	~12.9	~0.28	~15.25	~0.20	~21.35

Conclusion on thread usage

By using threads, the program gets successfully scaled. But, as we can see on the graph below, we cannot reach the linear S(p)/p relationship by increasing the amount of threads. At one moment, the thread increase gets progressively less useful for scaling.